

LECTURES – 16, 17, 18

DEPTH-FIRST SEARCH (DFS) OF A GRAPH

Another simple, basic algorithm, for both directed and undirected graphs.

Meaning of “depth first”:

From any vertex u , we follow the unexplored edges which lie on a path “outward” from it. When no such unexplored “outward” edges remain, we backtrack and find another such path.

Questions:

- You are in a totally unfamiliar city which has very irregular pattern of streets. Will you explore the streets by BFS or DFS?
- How about exploring a maze?
- Do we get a graph in a two person game, such as tic-tac-toe?

DFS (G)

```
// Initialization
for each vertex u in V[G]
    color(u) = WHITE;
    pred[u] = NIL;
time = 0;
// Search - as in [CLRS]
for each vertex u in V[G]      // *** MAIN LOOP ***
    if color[u] = WHITE
        DFS-VISIT(u);
```

```
// Alternate version
// Search with specified source vertex s
DFS-S(G, s)
    // Initialization
    for each vertex u in V[G]
        color(u) = WHITE;
        pred[u] = NIL;
    time = 0;
    DFS-VISIT(s);
```



```

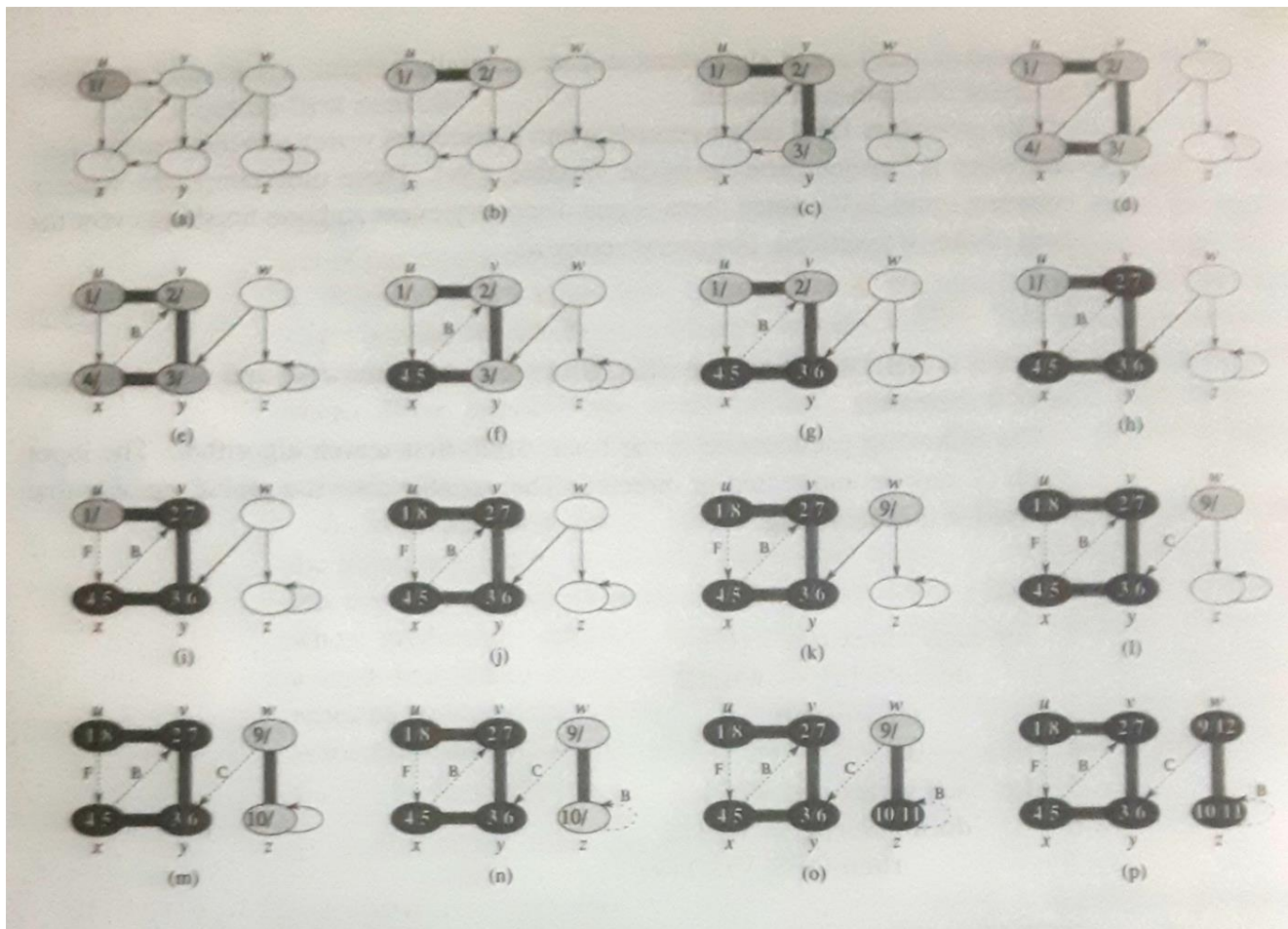
DFS-VISIT(u)
  color[u] = GRAY;
  time = time+1;
  d[u] = time; // discovery time

  for each vertex v in ADJ[u]
    // Search goes one step deeper
    if color[v] = WHITE
      pred[v] = u;
      DFS-VISIT(v);
  // We are done with node u
  color[u] = BLACK;
  time = time+1;
  f(u)= time;    // finish time

```

Running time analysis: Quite similar to that for BFS $\rightarrow O(V+E)$.

Example:



Provable properties of DFS

1. The [original version of] predecessor subgraph forms a forest of trees.
2. Suppose we represent the discovery and finish times of vertex u with the notation $(u$ and $u)$, respectively. Then the set of pairs $\{ (u, u) \mid \text{for all } u \text{ in } V \}$, taken in time order, define a **properly nested parenthesis structure**.

Further, for any two vertices u and v , exactly one of the following conditions holds:

- $d[u] \dots f[u]$ and $d[v] \dots f[v]$ are disjoint intervals, and neither vertex is a descendant of the other;
- $d[u] \dots f[u]$ is contained entirely within $d[v] \dots f[v]$, and u is a descendant of v ;
- $d[v] \dots f[v]$ is contained entirely within $d[u] \dots f[u]$, and v is a descendant of u ;

Classification of edges:

For directed graphs:

Tree edges: Edges in the depth-first forest; (u,v) is such an edge if vertex v was first discovered by exploring that edge.

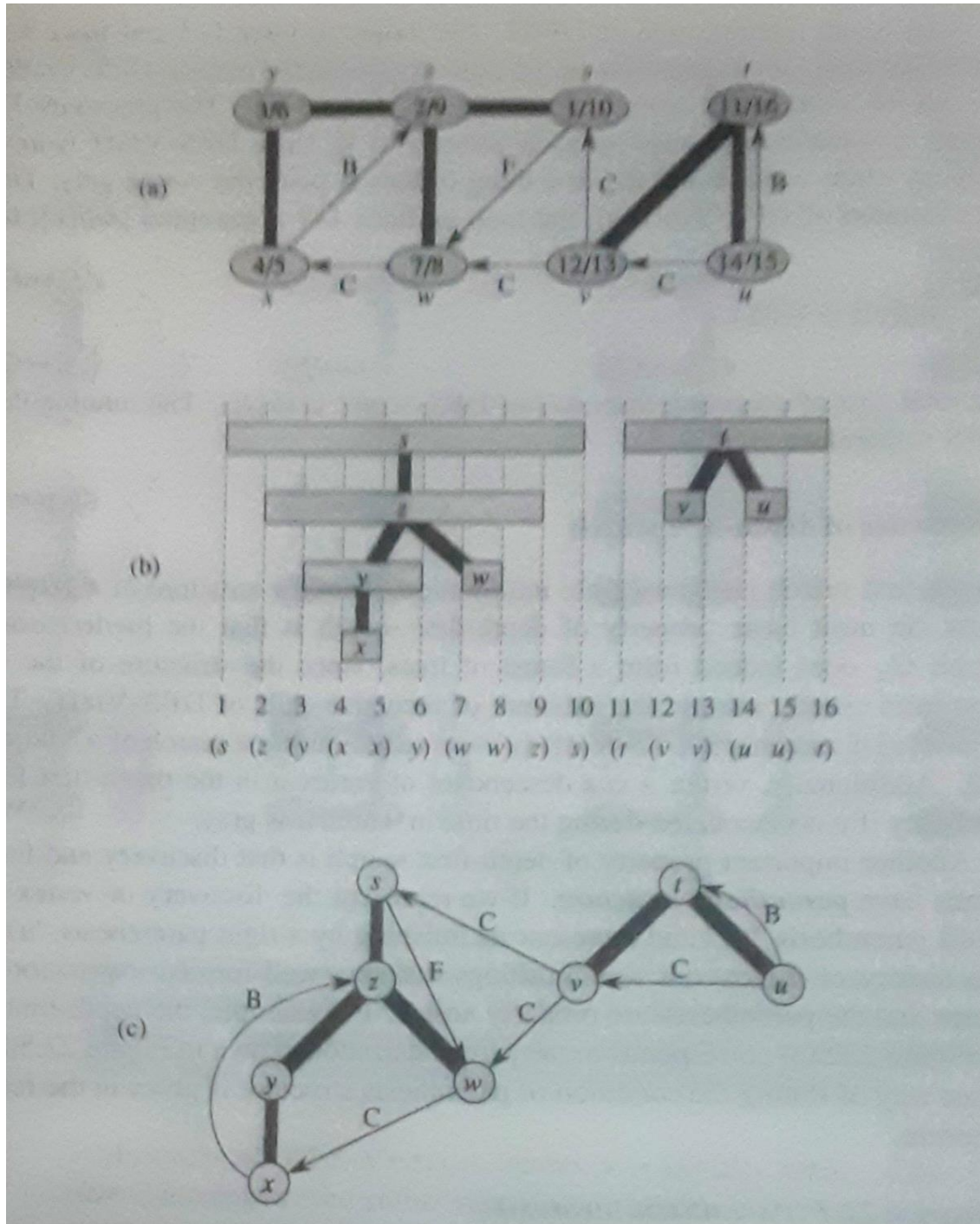
Back edges: Edges from a node to one of its ancestors in the depth-first tree, including self-loops in directed graphs.

Forward edges: Non-tree edges from a node to one of its descendants in the depth-first tree.

Cross edges: All other edges, which may lie within a depth-first tree, or across two trees in the depth-first forest.

For undirected graphs: Simply classify as tree edges and links (all other edges).

Another example:



DIRECTED, ACYCLIC GRAPH (DAG)

1. Consider the relation of **divisibility** between +ve integers. Can the graph of such a relation have a cycle of length > 1 ?
2. Consider the relation of pre-requisite between the sub-tasks of a project. Can the graph of such a relation have ANY cycle?

Simple examples.

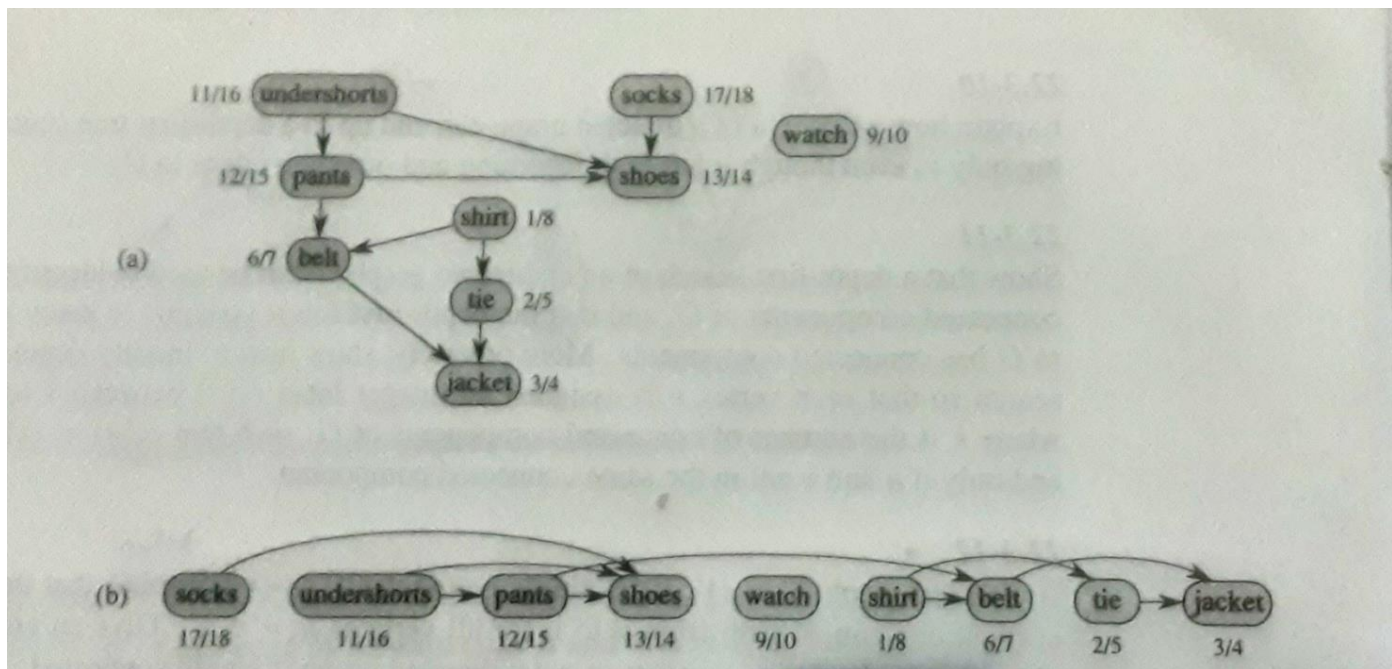
Meaning of **TOPOLOGICAL SORT** – find a total order consistent with DAG

Use of DFS in TOPOLOGICAL SORT

1. Using DFS, compute the finishing time $f[v]$ of each vertex v in V .
2. As each vertex is finished, add it to the front of a linked list.
3. Topologically sorted vertices appear in reverse order of their finishing times.

Example of algorithm: Figure 22.7

Dependence within the sub-tasks of a task.



STRONGLY CONNECTED COMPONENTS OF A DIRECTED GRAPH

Definition:

A **strongly connected component (SCC)** of directed graph $G = (V, E)$ is a **maximal set** of vertices $C \subseteq V$ such that, for every pair of vertices u and v in C , there is a path in C from u to v , and from v to u .

Meaning of 'maximal'.

Examples.

The transpose G^T of graph $G = (V, E)$ is defined by: $G^T = (V, E^T)$ where the set of edges $E^T = \{ (v, u) \mid (u, v) \text{ is in } E \}$. Informally, the edges of G^T are obtained by reversing the directions of the edges of G .

Examples.

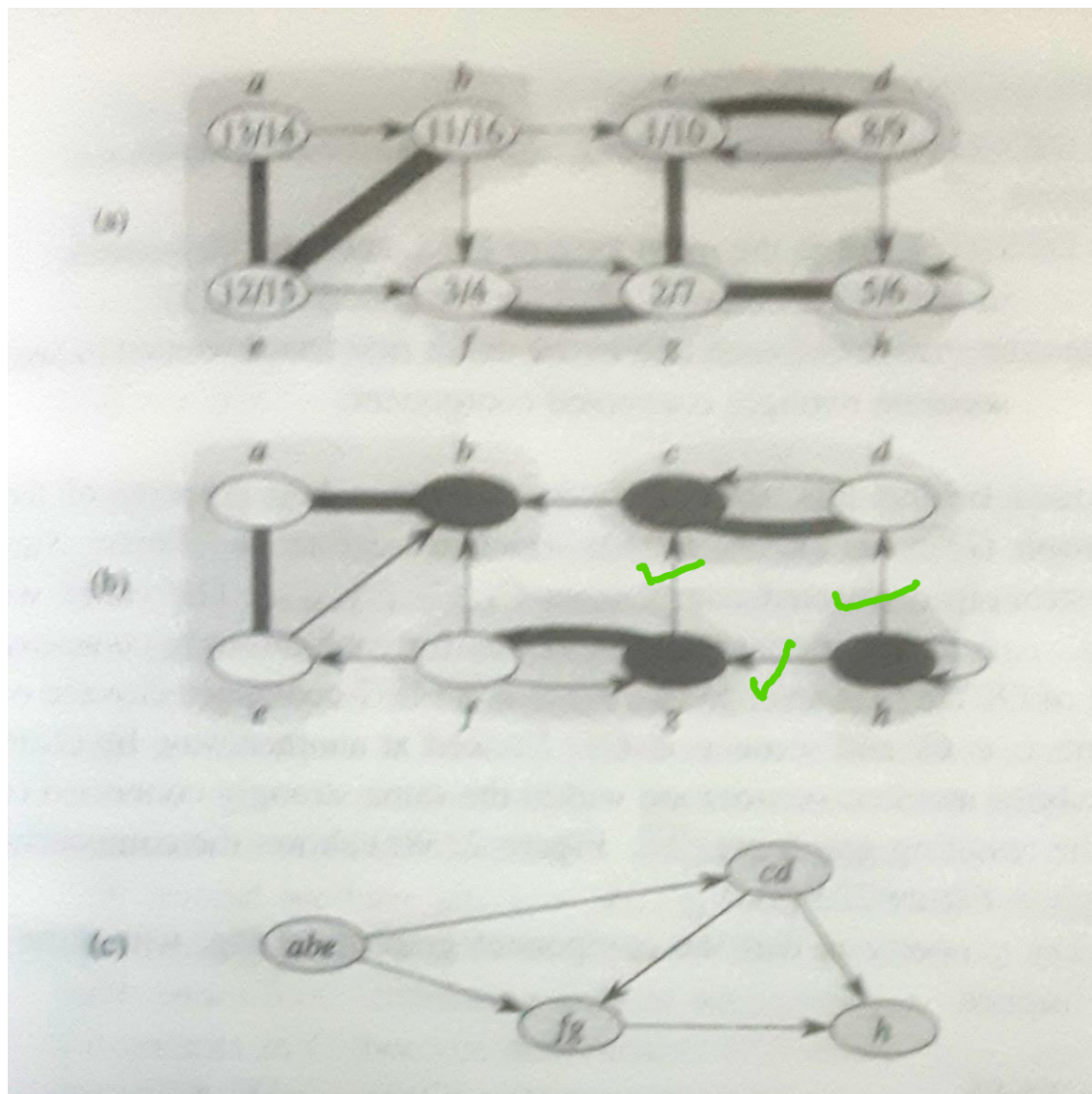
$O(V+E)$ algorithm to find strongly connected components:

SCC (G)

1. **DFS (G) ;**
 2. **Compute G^T ;**
 3. **DFS (G^T) SUCH THAT, IN THE *** MAIN LOOP ***,
VERTICES OF G ARE CONSIDERED IN
DECREASING ORDER OF $f[u]$ found in step 1 ;**
- RETURN (TREES OBTAINED IN STEP 3) ;**

Note the crucial step shown above in red, applied to the original DFS algorithm given in [CLRS].

Example of running the algorithm:



We will not go through the proof given in [CLRS], but these observations from the above example will clarify how the algorithm works:

1. The DFS forest of G – as seen in (a) – has TWO trees.
2. In the DFS forest of G^T , the same three vertices a,b,e form **a different tree**, because of the reversed edges in that SCC.
3. In the DFS forest of G^T , the earlier tree 'cdfgh' **breaks up into THREE trees**, because of the three reversed edges in G^T ticked in green.
4. The final four trees in G^T correspond to the four SCCs of G . Note that there are no cycles in the 'reduced' graph shown in (c).